UCES
UNIVERSIDAD DE CIENCIAS EMPRESARIALES Y SOCIALES

ACTIVIDAD OBLIGATORIA N°1

CARRERA:

Tecnicatura Universitaria en Programación de Sistemas

MATERIA:

Diseño Orientado a Objetos

BIMESTRE/CUATRIMESTRE Y AÑO

1° CUAT. /2026

PROFESOR/A:

Lic. Matias Velasquez

RESUMEN

Esta actividad obligatoria corresponde a la materia Diseño Orientado a Objetos (1° cuatrimestre 2026) de la Tecnicatura Universitaria en Programación de Sistemas. El objetivo del primer entregable es sentar las bases conceptuales y de diseño de un sistema orientado a objetos para un consultorio médico, a partir de información deliberadamente dispersa (mails, audios, fotos, notas y bocetos). El trabajo consiste en analizar, ordenar e interpretar los materiales para identificar requisitos funcionales y no funcionales, actores, casos de uso y un boceto inicial de clases, justificando las decisiones tomadas como en un entorno profesional.

La dinámica de trabajo se organiza en equipos de cuatro integrantes con roles rotativos (Analista de Requerimientos, Modelador de Casos de Uso, Diseñador de Clases Iniciales y Documentador/Coordinador), y exige evidencia de participación mediante Issues, ramas feature/, Pull Requests hacia develop, revisiones obligatorias y registro en changelog.md. La entrega se realiza exclusivamente mediante un repositorio público en GitHub con estructura y archivos definidos (README, anexos, introducción, diagramas con el MCP de Excalidraw e imagen exportada), y una Pull Request de release hacia master, cuyo enlace se publica tanto en el campus virtual como en Slack. La evaluación se basa en criterios formales y de contenido, con puntaje por rol (hasta 2,5 puntos por integrante) y requisitos mínimos de documentación y trazabilidad del trabajo colaborativo.

ÍNDICE

0 Indicaciones generales para los estudiantes....2
1. Requisitos formales de presentación de la actividad.... 3
1.1 Tiempo de desarrollo.... 4
1.2 Criterios de Evaluación....4
2 Consignas....4
2.1 Consigna del Primer Entregable - Diseño Orientado a Objetos.... 4
2.2 Problema a Resolver durante el Cuatrimestre....5
2.2.1 Archivos adjuntos para analizar....5
2.2.2 Herramienta para el análisis crítico....6
3 Dinámica práctica.... 6
4 Formato de Entrega....9
4.1 Pasos para entregar.... 12
4.2 Estructura de carpetas y archivos del proyecto....13
4.2.1 Archivos del proyecto propuestos....16
5 Puntaje por consigna....17
5.1 Tabla de evaluación (global + individual)....18
5.1.1 Ítems por Rol – cada uno vale hasta 2,5 puntos.... 18
Analista de Requerimientos (máx. 2,5 puntos).... 18
Modelador de Casos de Uso (máx. 2.5 puntos).... 19
Diseñador de Clases Iniciales (máx. 2,5 puntos)....19
Documentador y Coordinador (máx. 2,5 puntos)....19

0 Indicaciones generales para los estudiantes

Antes de comenzar a desarrollar la actividad, recuerde que el repositorio debe tener una portada en la que deben indicarse los siguientes datos de todos los integrantes del grupo: **nombre, número de matrícula, carrera y materia**.

La entrega se debe realizar en tiempo y forma en el espacio correspondiente, sin excepción. Es importante que verifiquen la fecha de entrega en el cronograma del aula.

Recuerde que para la aprobación de la materia deberá:

- **Aprobar dos exámenes parciales**: un primer parcial y segundo parcial (integrador) o,
- **Aprobar uno de los dos exámenes parciales** y **los recuperatorios**, si uno de ellos no hubiera sido aprobado o estuviera ausente.
- **Aprobar el examen final** proyectual y oral integrador.

La **nota mínima** de aprobación corresponde a **4 (cuatro)** puntos. La **calificación** se realizará mediante un puntaje, del 1 al 10, y/o a través de un porcentaje de aprobación.
Se valorará la participación activa en los foros, el cumplimiento con las actividades y la presencia en las conferencias web.

En caso de tener **inconvenientes técnicos** al momento de subir el archivo, realice captura de pantalla y remita a la casilla de Ayuda Técnica (**ayudatecnica@campusvirtual.uces.edu.ar**), detallando el número de matrícula, nombre y apellido y asignatura en que sucedió el inconveniente, adjuntando:

- **El print de pantalla si existiese un problema propio del campus**. Es fundamental que sea claro y muestre toda la información: **horario, fecha, intento, error**. Le sugerimos que utilice el botón "Impr Pant Pet Sis". Sin este registro la instancia no se contabilizará y se lo calificará como **Ausente o Desaprobado** según corresponda.
- **El número de reclamo en caso de ser un problema de luz/conectividad propia del alumno**. Sin este registro la instancia no se contabilizará y se lo calificará como **Ausente**.

No se considerará entregada ni se evaluará aquella actividad que sea enviada por fuera del espacio correspondiente, como tampoco la que no se encuentre completa y/o corresponda a otra asignatura.

Una vez que el área reciba esta información se analizará en profundidad y **dentro de las 48hs hábiles se brindará la respuesta correspondiente.**

1. Requisitos formales de presentación de la actividad

Dado que la actividad se realizará utilizando GitHub como plataforma principal para la entrega y desarrollo del trabajo, se solicita que en el campus virtual solo se presente un enlace a la pull request con la release de la entrega final de la actividad del repositorio correspondiente. No se aceptarán archivos adjuntos ni documentos adicionales que no correspondan a lo solicitado, ya que parte integral de la actividad es utilizar esta herramienta colaborativa para gestionar y compartir el trabajo realizado. Por lo tanto, el enlace proporcionado deberá dirigirse al repositorio de GitHub que

contenga todo el contenido relacionado con la actividad, incluyendo los diagramas, documentos y cualquier otra información relevante. Por ejemplo: https://github.com/TuUsuario/Nombre-del-Repositorio/pull/1

IMPORTANTE: Sin pull request enlazada en el changelog.md no hay justificación de trabajo y participación en el proyecto, por lo que estará desaprobado.

1.1 Tiempo de desarrollo

Tiempo Límite para la Actividad:

- Fecha de Habilitación de Consignas: 09 de marzo, a las 08:00
- Fecha Límite de Entrega: 26 de marzo, a las 23:55

Tiempo Total para Realizar la Actividad:

- El tiempo total para realizar la actividad es de 15 días desde la fecha de habilitación de consignas hasta la fecha límite de entrega.

Temporizador y Número de Intentos:

- Se otorga un único intento para realizar la actividad.
- No hay un temporizador específico para esta actividad, pero se espera que los estudiantes la completen dentro del período de 15 días asignado.

1.2 Criterios de Evaluación

- ✔ Comprensión e interpretación de la consigna formulada.
- ✔ Dominio adecuado de los temas abordados.
- ✔ Aplicación de la teoría a situaciones prácticas.
- ✔ Manejo del vocabulario específico.
- ✔ Pertinencia y claridad en la redacción.
- ✔ Estilo académico, correcta construcción gramatical, puntuación y acentuación.
- ✔ Cumplimiento de las pautas de presentación de trabajos escritos.

2 Consignas

2.1 Consigna del Primer Entregable - Diseño Orientado a Objetos

Objetivo: Sentar las bases conceptuales y de diseño para el sistema que desarrollaremos durante el cuatrimestre, aplicando programación orientada a objetos (POO) y buenas prácticas de trabajo colaborativo con herramientas como GitHub, plantuml y asistentes de IA.

2.2 Problema a Resolver durante el Cuatrimestre

Durante este cuatrimestre, trabajaremos en el diseño orientado a objetos de un sistema realista para un pequeño consultorio médico que busca digitalizar la gestión de sus turnos.

El trabajo no parte de un documento de requerimientos cerrado, sino de varios archivos sueltos (mails de clientes, notas de reuniones, mensajes de chat, bocetos en PDF, etc.).

Su primer desafío será leer, analizar e interpretar esa información para identificar los requisitos funcionales y no funcionales, actores, casos de uso y posibles clases del sistema. Como en un entorno profesional, tendrán que construir la especificación del sistema a partir de la información dispersa que el “cliente” (el profesor) les proveerá, justificando sus decisiones y documentando cada paso.

2.2.1 Archivos adjuntos para analizar

⚠️Todos estos archivos están “desordenados” a propósito: es parte de la actividad ordenarlos, interpretar y extraer requerimientos funcionales.

01 - Mail de la secretaria.pdf

02 - Transcripción Gemini - Google Meet - KickOff.pdf

03 - Audio WhatsApp - Dr. Molina.mp3

04 - foto_cuaderno_respuestas_dr_molina-parte-1.png

05 - foto_cuaderno_respuestas_dr_molina-parte-2.png.png

06 - 2026-03-15_slack_#canal-proyecto-turnos-consultorio_equipo_arranque_mvp.txt

📄 07 - Transcripción Gemini - Google Meet - AyD.pdf

08 - 2026-03-16_slack_#canal-proyecto-turnos-consultorio_boceto_inicial_clases

📄 09 - Audio WhatsApp - Secretaria Valeria.mp3

📄 10 - Transcripción Gemini - Google Meet - ajuste de alcance MVP.pdf

2.2.2 Herramienta para el análisis crítico

Para facilitar la lectura, síntesis y organización de la información, deben apoyarse en Notebook LM **un asistente de investigación con inteligencia artificial gratuito**. Esta herramienta puede ayudarles a resumir textos largos, proponer posibles requisitos o generar tablas comparativas de actores y casos de uso, clases candidatas, etc.

💡 ▶ NotebookLM: la nueva forma de estudiar y organizar tus ideas con IA

⚠️ Recuerden que el análisis final debe ser **crítico y propio**: la IA puede ayudar a explorar ideas o visualizar estructuras, pero las decisiones de diseño, redacción y justificación deben surgir del grupo y serán evaluadas como parte del trabajo.

3 Dinámica práctica

La dinámica de trabajo deberá seguir estas pautas:

- Los grupos estarán formados por **4 integrantes**, quienes deberán asignarse roles específicos en cada entrega, rotándolos para que cada estudiante experimente distintas responsabilidades.
- Los roles para esta entrega son:
  - **Analista de Requerimientos:**
    1. Se encarga de estudiar los archivos sueltos (mails, audios, fotos, notas) y extraer los requisitos funcionales y no funcionales.
    2. Compartir el cuaderno creado en Notebook LM con todos los integrantes del grupo. Incluir enlace del cuaderno en la sección “Requisitos iniciales del sistema”.
    3. Documenta estos requisitos en la introducción.md (sección “Requisitos iniciales del sistema”). Redacta y enumera correctamente los mismos. Por ejemplo: RF1, RNF1, etc.

4. Crea Issue con el título "A1 - Análisis de requerimientos", incluyendo los pasos que implicó su tarea en la descripción de la Issue. Vincula esta issue con la PR asociada, y se asegura que la issue se cierre cuando la PR este mergeada.
5. PR asociado: agrega o actualiza el archivo introduccion.md con los requisitos.

- **Modelador de Casos de Uso**
  1. Identifica actores y casos de uso a partir de los requisitos.
  2. Documenta cinco casos de uso completos con todos los campos solicitados.
  3. Se asegura que estos casos de uso coincidan con el cuaderno Notebook LM compartido.
  4. Crea Issue con el título "A1 - Modelado de Casos de Uso", incluyendo los pasos que implicó su tarea en la descripción de la Issue. Vincula esta issue con la PR asociada, y se asegura que la issue se cierre cuando la PR este mergeada.
  5. PR asociado: agrega o actualiza los casos de uso en introduccion.md
- **Diseñador de Clases Iniciales:**
  1. Diseña el boceto inicial de clases basado en los requisitos y casos de uso.
  2. Realiza el diagrama en Excalidraw (haciendo uso del MCP en VSCode), genera imagen y lo sube a la carpeta diagramas/.
  3. Se asegura que este boceto de clases coincide con el cuaderno Notebook LM compartido.
  4. Crea Issue con el título "A1 - Diseño de clases iniciales", incluyendo los pasos que implicó su tarea en la descripción de la Issue. Vincula esta issue con la PR asociada, y se asegura que la issue se cierre cuando la PR este mergeada.
  5. PR asociado: agrega o actualiza boceto_inicial_clases.puml y su imagen.
- **Documentador y Coordinador de Repositorio**
  1. Crea la estructura de carpetas y archivos en el repositorio siguiendo el formato definido. Se asegura de que todo el proyecto cumpla el formato de entrega correcto en markdown.

2. Redacta README.md y anexos.md con enlaces y explicaciones claras. Completa las secciones “Descripción del paradigma orientado a objetos” y “Los cuatro fundamentos de POO” en introduccion.md
3. Crea Issue con el título “A1 - Documentador y Coordinador del repositorio”, incluyendo los pasos que implicó su tarea en la descripción de la Issue. Vincula esta issue con la PR asociada, se asegura que la issue se cierre cuando la PR este mergeada. Por otro lado, administra Issues, asegurándose de que cada integrante las haya creado correctamente y vinculadas en cada PR asociada.
4. Administra Pull Requests para que cada tarea esté documentada y revisada. Se requiere al menos una revisión por pull request del proyecto asistida con IA (**4 mínimo**), determinando cuales comentarios son pertinentes, y cargando los request changes en las líneas correspondientes. Esta revisión no solo es estructural o sintáctica, sino que asegura la calidad del diseño cumpla como con los Requerimientos Funcionales del proyecto, extraídos en el cuaderno de Notebook LM.

   💡 Code Review con IA GRATIS en GitHub y VS Code — Tutorial completo paso a paso

5. PR asociado: agrega o actualiza README.md, anexos.md, introduccion.md y estructura de carpetas.

- Cada integrante abre Issues para las tareas que va a realizar.
- Trabajan en ramas propias, nombradas con el formato: **feature/<rol>-<descripción>**.
- Hacen PRs a la rama **develop**, vinculando los Issues.
- En el changelog.md registran quién hizo qué PR y resumen el aporte concreto, incluyendo el link a la PR y al Issue.
- La rama **master** estará protegida y sólo se integrarán cambios desde develop (por medio de la release) luego de revisión y aprobación por parte del **profesor**.

- La rama **develop** estará protegida y sólo se integrarán cambios desde una feature luego de revisión y aprobación por parte del **Documentador y Coordinador de Repositorio.**

  💡 ¿Como configurar GitHub? - Reglas de protección de rama

4 Formato de Entrega

- **Repositorio en GitHub:** Crear un repositorio público llamado: `SistemaTurnosMedicos`. Invitar al usuario MVelasquez98 como colaborador del repositorio, así como a todos los integrantes del equipo.
- **Ramas en repositorio:**
  - **develop:** esta rama funciona como el espacio común de trabajo del equipo. Es donde integran sus avances individuales antes de preparar la entrega oficial en una rama `release`.
    Utiliza los siguientes comandos en git para crear la rama

    ```
    git checkout -b develop
    git push -u origin develop
    ```

    git checkout -b develop: crea una nueva rama local llamada develop y te posiciona en ella.
    git push -u origin develop: sube la rama al repositorio remoto y establece el seguimiento.

  - **master:** es la rama principal del proyecto. Contiene la versión estable y oficial del código, lista para usar o publicar. Solo recibe cambios que ya fueron probados y aprobados, provenientes de una rama `release`..

    Utiliza los siguientes comandos en git para crear la rama

    ```
    git checkout -b master
    git push -u origin master
    ```

    git checkout -b master: crea una nueva rama local llamada master y te posiciona en ella.
    git push -u origin master: sube la rama al repositorio remoto y establece el seguimiento.

La rama `master` está protegida. No podrán hacer merge de la `release` hasta que el docente apruebe formalmente la Pull Request en GitHub con el LGTM y approve.

- **feature:** cada participante del grupo debe colaborar en *al menos una* rama individual, de acuerdo al rol asignado para la actividad. Por ejemplo:
  - **Analista de Requerimientos:** feature/analista-requerimentos-add-introduccion-md
  - **Modelador de Casos de Uso:** feature/modelador-casos-uso-update-use-case-1
  - **Diseñador de Clases Iniciales:** feature/diseniador-clases-add-boceto-inicial
  - **Documentador y Coordinador de Repositorio:** feature/doc-coord-repo-update-readme-md

Cada integrante debe crear su propia rama `feature/` a partir de `develop` usando el comando:

```
git checkout -b feature/nombre-de-la-rama develop
```

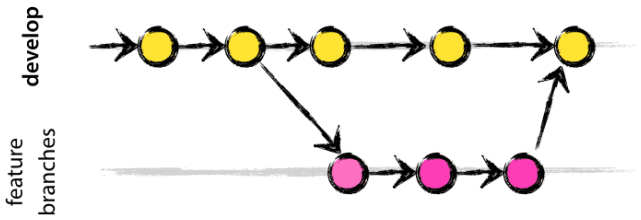
Una vez finalizada la tarea asignada, deben subir los cambios con:

```
git add .
git commit -m "Descripción clara del cambio realizado"
git push origin feature/nombre-de-la-rama
```

Luego, deben crear una Pull Request (PR) en GitHub desde su rama `feature/` hacia la rama `develop`, explicando brevemente qué se hizo. Utiliza esta plantilla para la pull request, recorda que debes automatizar el uso de la plantilla en el repositorio para el uso de todos los colaboradores.

El integrante del grupo con el rol **Documentador y Coordinador de Repositorio** debe revisar y aprobar la PR antes de hacer el *rebase and merge.* Esto permite mantener un control de los cambios y practicar el trabajo colaborativo en equipo.

Git Branching Model propuesto:



💡 ¿Cómo usar GitHub? - ¡Todo lo que necesitás saber!

💡 ¿Cómo usar GitFlow? - Primeros pasos

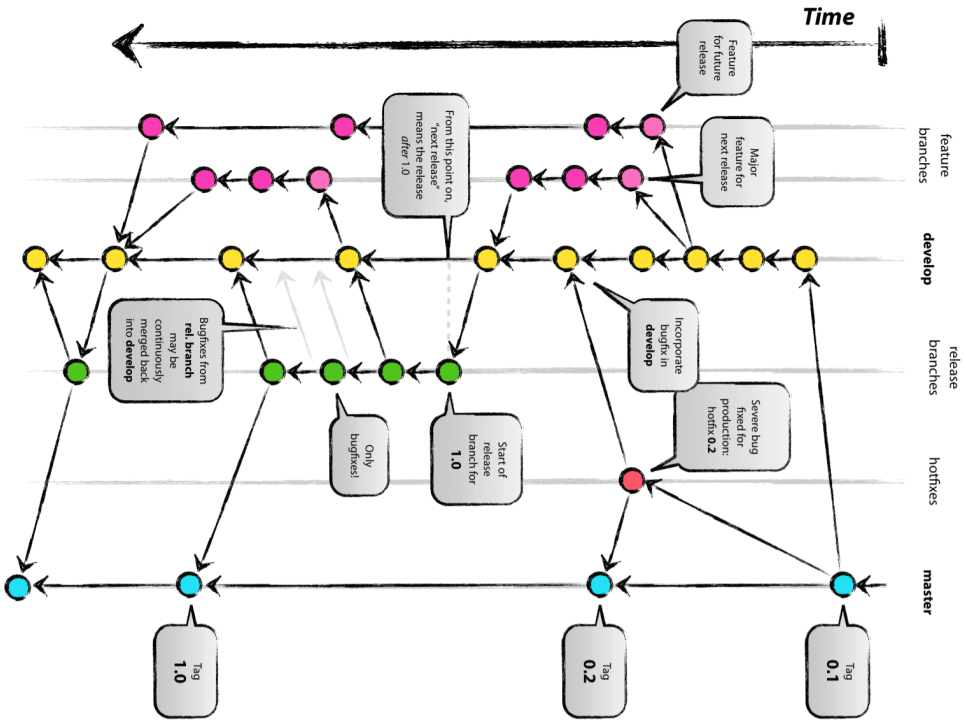
- **release/actividad-obligatoria-1:** Esta es la rama que representa la versión entregable del trabajo del grupo. Se trata de una rama especial donde se integran todos los cambios definitivos realizados en `develop`, listos para ser evaluados. Al finalizar la integración de todas las tareas individuales en `develop`, quien tenga el rol de Documentador y Coordinador de Repositorio debe:
  - Crear una nueva rama desde develop:

    ```
    git checkout develop
    git pull
    git checkout -b release/actividad-obligatoria-1
    ```

  - Subir esta nueva rama al repositorio remoto:

    ```
    git push origin release/actividad-obligatoria-1
    ```

Git branching model propuesto:



4.1 Pasos para entregar

1. Ir al repositorio en GitHub.
2. Abrir una Pull Request (`Compare & pull request`) de `release/actividad-obligatoria-1` hacia `master`.
3. En el título de la Pull Request, usar el siguiente formato:

   `[GRUPO N°X][Actividad Obligatoria N°1] Entrega de la actividad`

4. En la descripción, utilizar el template de pull request proporcionado para las release branches. Recorda, que podes contar con diferentes plantillas de pull request, como por ejemplo en este caso, para las feature branches y las release branches.

5. Se entrega por el campus virtual en la actividad correspondiente solamente este enlace referente a la pull request de la release.

6. Compartir el link de la Pull Request en el canal de Slack `#2026-1er-cuat-doo`, mencionando en hilo de conversación a `@Matias Velasquez`. Utilizar mismo formato que para el titulo de la pull request.

7. ¿Qué pasa después? Se realizará una revisión técnica (code review) por parte del docente. Se usarán estos emojis para el seguimiento en el hilo de Slack:
   - 💬 **:speech_balloon:** Comentarios u observaciones del docente o equipo, pendientes de revisión.
   - ✅ **:white_check_mark:** Correcciones realizadas por el grupo.
   - LGTM **:lgtm:** Aprobación final del docente en GitHub (Looks Good To Me).
   - **:pr-merged:** Merge completado (lo debe notificar el grupo una vez realizado).

8. Una vez aprobado:
   - Un integrante autorizado del grupo hace el merge pull request desde GitHub (preferentemente quien tenga el rol de **Documentador y Coordinador** de Repositorio).
   - El grupo debe notificarlo en el hilo de conversación de la entrega en Slack con el emoji **:pr-merged:** . El merge también puede formar parte de la **Actividad Obligatoria N°2**, ya que incluye las correcciones posteriores a la revisión.

4.2 Estructura de carpetas y archivos del proyecto

```
SistemaProductoraVideos/
├── README.md
├── changelog.md
├── diagramas/
│   └── 01-diagrama-clases/
│       ├── 01-boceto-inicial.excalidraw
│       └── 01-boceto-inicial.png
└── anexos/
    ├── anexos.md
    └── introduccion.md
```

- **README.md:** Breve descripción del proyecto, objetivo del entregable, y enlace a `anexos/anexos.md`, en un listado de viñetas bajo el título “Diagramas y Diseños”. Con el pasar de las entregas, se agregaran más items al listado. No

olvidar incluir la carátula del proyecto en este apartado, con los datos necesarios de cada integrante del grupo: nombre, número de matrícula, carrera y materia.

💡 ▶️ Markdown desde Cero en 10 Minutos 🚀 Guía Rápida

- **changelog.md:** Este archivo tendrá la función de registrar, entrega por entrega, qué cambios se hicieron, quiénes participaron, qué roles asumieron y los enlaces a las Pull Requests (PR) e Issues que permitieron integrar dichos cambios. Usar esta plantilla. Cada integrante debe actualizar este archivo con cada Pull Request desde su rama `feature/`. No olviden incluir su contribución para mantener el control de los cambios. El revisor de cada PR debe asegurarse de que la sección correspondiente en changelog.md esté correctamente completada antes de aprobar.

  IMPORTANTE: Sin pull request enlazada en el changelog.md no hay justificación de trabajo y participación en el proyecto, por lo que estará desaprobado.

- **anexos.md:** Archivo que funciona como índice de los anexos existentes en la carpeta `anexos/`. Incluir el título “Anexos” al archivo e incluir una lista no ordenada markdown que contenga todos los enlaces a los anexos que se irán agregando en futuras entregas. No olvidar incluir un ítem que dirija a otro archivo llamado `introduccion.md`, dentro de la misma carpeta `anexos/`.

- **introduccion.md:** Debe contener lo siguiente:
  - **Descripción del paradigma orientado a objetos:** Definir qué es el paradigma orientado a objetos y por qué es importante.
  - **Los cuatro fundamentos de POO:** Describir con ejemplos del proyecto cómo se podrían aplicar estos principios en el diseño. Usar diagramas o bocetos simples, no es necesario UML ni código en esta etapa aún.
  - **Requisitos iniciales del sistema:** Describir de forma completa al menos cinco requisitos funcionales, y cinco no funcionales.
  - **Casos de uso:** Desarrollar cinco casos de uso con el formato adecuado:
    - Nombre del caso de uso
    - Actor(es) involucrado(s)

    - Descripción breve
    - Flujo principal de eventos (no menos de 5 pasos)
    - Precondiciones
    - Postcondiciones

- **Boceto inicial del diseño de clases:** Realizar un boceto inicial del diseño de clases utilizando MCP Excalidraw y la Extensión de Excalidraw en VSCode. Este diagrama debe incluir al menos los elementos básicos de una clase: nombre, atributos, métodos (comportamiento) y relaciones básicas que identifiques inicialmente. Se debe:
    - Crear un archivo `01-boceto-inicial.excalidraw` en la carpeta subcarpeta `01-diagrama-clases/` dentro de la carpeta raíz `diagramas`, con el código fuente del diagrama.
    - Generar la imagen `01-boceto_inicial_clases.png` a partir del archivo .excalidraw., y también almacenarla en la carpeta subcarpeta `01-diagrama-clases/` dentro de la carpeta raíz `diagramas`

💡 ▶️ La IA dibuja tus diagramas en VS Code 🤯 Excalidraw MCP en 5 mi…

4.2.1 Archivos del proyecto propuestos

`README.md` propuesto

Nombre del Proyecto - Diseño Orientado a Objetos

- **Materia:** Diseño Orientado a Objetos
- **Carrera:** Tec. en Programación de Sistemas
- **Grupo:** Grupo N° X

Introducción

Breve descripción del proyecto:
Este proyecto tiene como objetivo desarrollar el diseño orientado a objetos de una aplicación para [...]. La solución busca resolver [...].

Integrantes:

Nombre y Apellido	Matrícula	Usuario de GitHub
Nombre Estudiante 1	XXXXX	@username
Nombre Estudiante 2	XXXXX	@username
Nombre Estudiante 3	XXXXX	@username
Nombre Estudiante 4	XXXXX	@username

Diagramas y Diseños

- Anexos

`anexos.md` propuesto

Anexos

- Anexo - Introducción al Diseño Orientado a Objetos

`introduccion.md` propuesto

Anexo - Introducción al Diseño Orientado a Objetos

Definir qué es el paradigma orientado a objetos y por qué es importante..

Los cuatro fundamentos de POO

Describir con ejemplos del proyecto (Usar diagramas o bocetos simples, no es necesario UML ni código en esta etapa)

Requisitos iniciales del sistema

Describir al menos cinco requisitos funcionales, y cinco no funcionales.

Casos de uso

Desarrollar cinco casos de uso con el formato adecuado

Boceto inicial del diseño de clases

Incluir una imagen incrustada en el archivo y un enlace para visualizarla en línea.

5 Puntaje por consigna

- Cada estudiante aporta hasta **2,5 puntos** al trabajo grupal, según su rol.
- Cada rol tiene **criterios detallados**, que permiten una corrección rápida, objetiva y basada en evidencia.
- El resultado del grupo es **una única nota común**, ajustando la nota individual según performance.
- En caso de contingencias (abandono, poca participación), el equipo debe adaptarse y asumir los roles necesarios, y documentar estos hechos en el `changelog.md`
- En caso de nula o muy baja participación de un integrante, o falta de evidencia en el `changelog.md`, ese estudiante podrá no aprobar la actividad, aunque el grupo obtenga una buena calificación.
- La documentación de tareas en el `changelog.md` es **obligatoria**. Es responsabilidad del grupo registrar adecuadamente el trabajo realizado por cada integrante.
- Si un integrante asume más de un rol por contingencias reales (abandono, enfermedad, etc.), y eso queda claramente documentado, podrá obtener un reconocimiento extra por su esfuerzo, sin que esto signifique una sobrecarga: el trabajo está diseñado para poder ser completado incluso por una sola persona, si fuese necesario.

- Las **weekly meetings** (encuentros semanales) son el espacio formal para plantear dificultades, reorganizar el equipo, y —si corresponde— solicitar intervención docente.

5.1 Tabla de evaluación (global + individual)

Estudiante	Rol Asignado	Puntaje Rol (máx. 2,5)
	Analista de Requerimientos	
	Modelador de Casos de Uso	
	Diseñador de Clases Iniciales	
	Documentador y Coordinador	

5.1.1 Ítems por Rol – cada uno vale hasta 2,5 puntos

Analista de Requerimientos (máx. 2,5 puntos)

- ☐ **[0,5 puntos]** Extrajo mínimo 5 requisitos funcionales completos.
- ☐ **[0,5 puntos]** Extrajo mínimo 5 requisitos no funcionales completos.
- ☐ **[0,5 puntos]** Documentó claramente los requisitos en `introduccion.md`, incluyendo enlace a cuaderno grupal de Notebook LM.
- ☐ **[0,5 puntos]** Abrió Issue para su tarea asignada, con descripción, y se vinculó con PR.
- ☐ **[0,5 puntos]** Participación técnica**:**
    - ☐ Creó `feature/` branch.
    - ☐ Documentó en `changelog.md`.
    - ☐ Hizo PR y merge revisado por el **Documentador y Coordinador**.

Modelador de Casos de Uso (máx. 2.5 puntos)

- ☐ **[0,5 puntos]** Identificó 5 casos de uso relevantes, haciendo uso de Notebook LM.
- ☐ **[0,5 puntos]** Documentó todos los campos: nombre, actores, descripción, flujo principal, pre/postcondiciones.
- ☐ **[0,5 puntos]** Registró los casos en `introduccion.md`.
- ☐ **[0,5 puntos]** Abrió Issue para su tarea asignada, con descripción, y se vinculó con PR.
- ☐ **[0,5 puntos]** Participación técnica**:**
  - ☐ Creó `feature/` branch.
  - ☐ Documentó en `changelog.md`.
  - ☐ Hizo PR y merge revisado por el **Documentador y Coordinador**.

Diseñador de Clases Iniciales (máx. 2,5 puntos)

- ☐ **[0,5 puntos]** Creó el diagrama en `.excalidraw` con clases, atributos y métodos básicos.
- ☐ **[0,5 puntos]** Reflejó relaciones mínimas entre clases.
- ☐ **[0,25 puntos]** Exportó imagen `.png` correctamente
- ☐ **[0,25 puntos]** Abrió Issue para su tarea asignada, con descripción, y se vinculó con PR.
- ☐ **[0,5 puntos]** Guardó archivos en `diagramas/01-diagrama-clases/`.
- ☐ **[0,5 puntos]** Participación técnica**:**
  - ☐ Creó `feature/` branch.
  - ☐ Documentó en `changelog.md`.
  - ☐ Hizo PR y merge revisado por el **Documentador y Coordinador**.

Documentador y Coordinador (máx. 2,5 puntos)

- ☐ **[0,25 puntos]** Armó correctamente la estructura de carpetas y archivos.
- ☐ **[0,25 puntos]** Se aseguró de que se respete el formato de entrega markdown en todo el proyecto. Se aseguró que se utilizaron los templates correspondientes proporcionados para las pull request y `changelog.md`.

- ☐ **[0,5 puntos]** Completó `README.md`, `anexos.md`, e introducción a POO en `introduccion.md`.
- ☐ **[0,25 puntos]** Abrió Issue para su tarea asignada, con descripción, y se vinculó con PR. Coordinó Issues del equipo, asegurándose que todas estén creadas, vinculadas con PR correspondiente y cerradas post-merge de la PR.
- ☐ **[0,5 puntos]** Coordinó las PRs y realizó mínimo 4 revisiones de pull request asistidos con IA, de `feature/` branches contra `develop` por parte del resto de integrantes del equipo; cargando las request changes correspondientes.
- ☐ **[0,25 puntos]** Se usaron correctamente ramas (`feature/`, `develop`, `release/`).
- ☐ **[0,5 puntos]** Participación técnica**:**
  - ☐ Creó `feature/` branch y `release/`branch para la entrega final.
  - ☐ Realizó entrega de la misma en el campus y publicó en slack.
  - ☐ Documentó en `changelog.md`.
  - ☐ Hizo PR y merge revisado por otro integrante del equipo.

Total: 10 puntos

Nota: Los entregables siguientes se construirán sobre esta base, extendiendo y refinando el diseño del sistema. Si algún ítem no se cumple o está incompleto, se dejará marcado como *"Request Changes"* para su corrección en la **Actividad 2**.